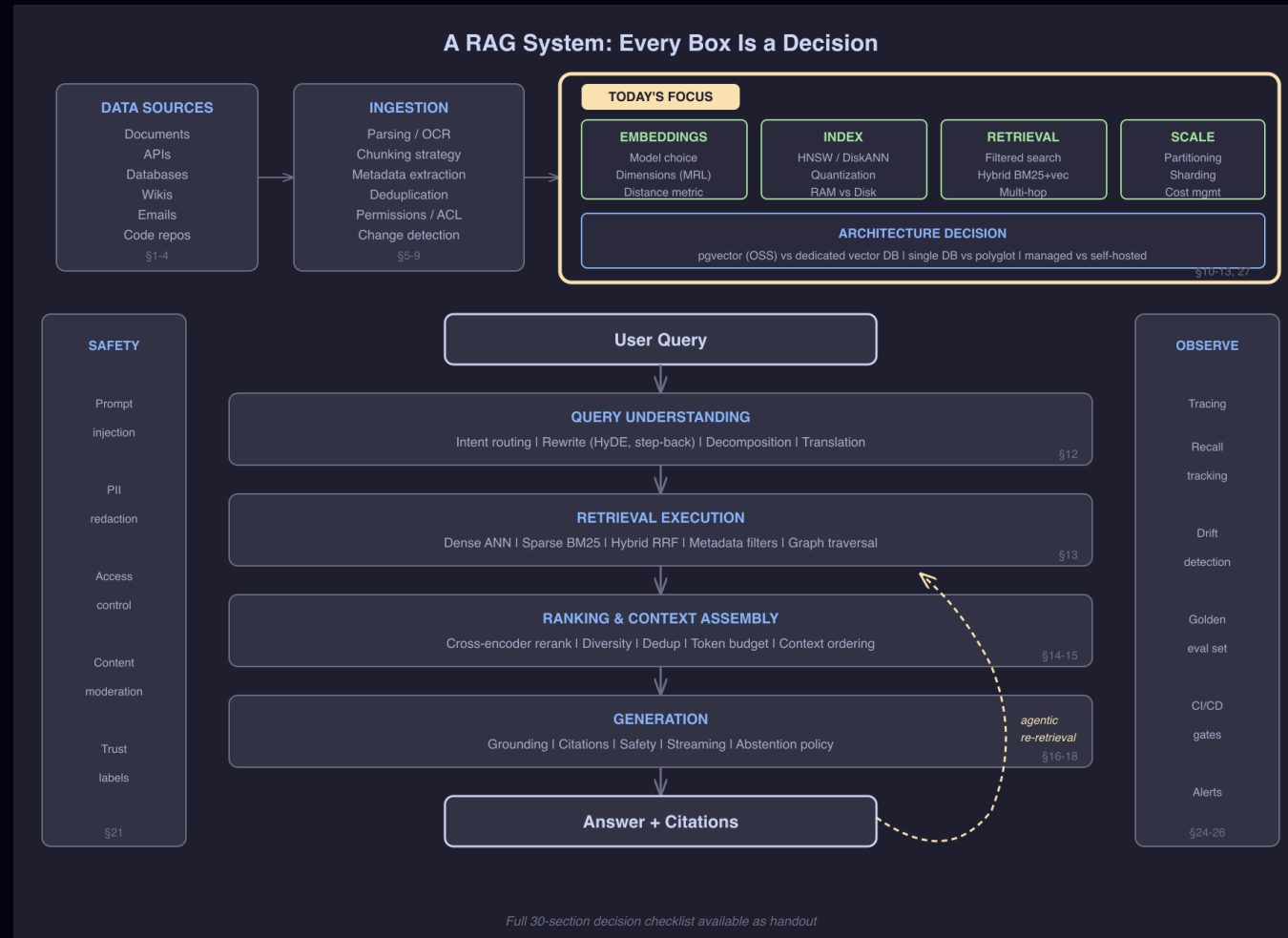


Storing High-Dimensional Data at Scale

The RAM Wall, Quantization, Filtered Search & When to Use What

Narrated by: Jeevan

April 2026



Why This Talk, Why Now

```
Month 1: pgvector, 100K vectors ✓  
Month 4: 10M vectors, still fine ✓  
Month 8: 100M vectors, RAM explodes 💣  
Month 9: Per-tenant filtering → recall 40% 📉  
Month 10: "Add Pinecone?" → sync forever 🔄
```

Same pattern. Same order. Every team.

This talk: the math, the tools, and the decisions — in plain English.



■ Today's Journey

■ First Principles

Embeddings, HNSW, the RAM wall

■ Three Compression Levers

Dimensions → Bits → Disk

■ Silent Failures

Filtered search + recall drift

■ Architecture Decisions

Benchmarks, trade-offs, decision matrix

How Do Computers Compare Text?

The fundamental problem: computers don't understand words.

"I love fries" vs "Fries are great"

🧑 Human: instantly knows these are similar. 🖥️ Computer: two completely different strings.

The fix: turn text into numbers that capture meaning.

```
"I love fries"      → [0.2, 0.8, 0.1, ...] 384 numbers  
"Fries are great"   → [0.3, 0.7, 0.2, ...] 384 numbers  
"The sky is blue"   → [0.9, 0.1, 0.8, ...] 384 numbers
```

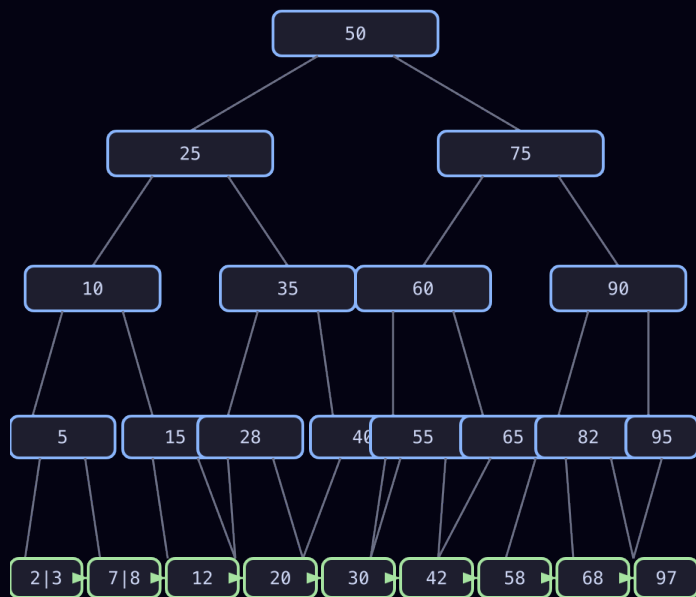
Similar meanings → similar numbers.

That's an embedding. A coordinate in "meaning space."



Why Vector Indexing Is Different

B-tree: exact ordering, binary search, $O(\log n)$



B+ Tree – sorted keys, $O(\log n)$ lookup
Leaf-linked for range scans. Binary search works
because there's a natural order.

Vector indexes have no natural sort order.

```
A = [0.21, 0.87, 0.14, ..., 0.53]
B = [0.93, 0.12, 0.38, ..., 0.71]
C = [0.34, 0.76, 0.22, ..., 0.48]
```



Which comes first – A or B? There's no "less than" for 1536 dimensions.

Exact search = check every vector. That's too slow at scale.

But what if we don't need the exact closest – just **close enough**?

■ How ANN (Approximate Nearest Neighbor) Indexes Work

We need "close enough" without checking everything.

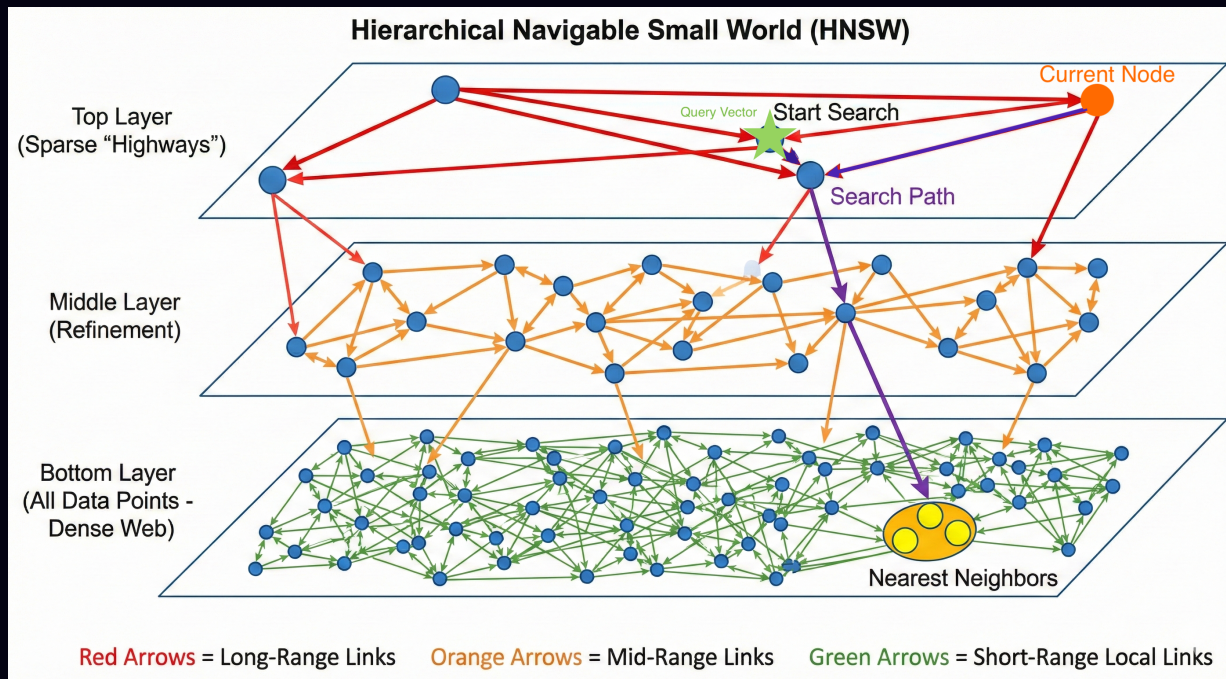
HNSW

Multi-layer navigable graph.

"GPS: highways first, then local roads."

~95-99% recall, 1-5ms latency.

Lives entirely in RAM.



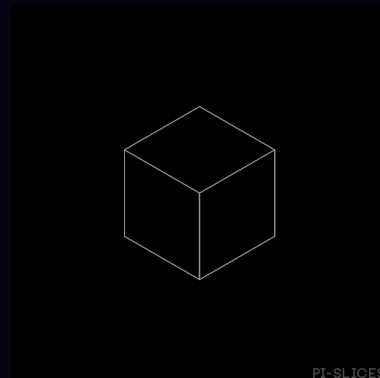
Everyone wants to build AI on their own data. Leadership wants to know why the infrastructure bill just tripled.



Per vector: $1536 \text{ dims} \times 4 \text{ bytes} = 6,144 \text{ bytes} \approx 6 \text{ KB}$

Scale	Raw Vectors	+ HNSW (50%)	Approx. RAM Cost
1M	6 GB	~9 GB	~\$50/mo
10M	61 GB	~92 GB	~\$500/mo
100M	614 GB	~920 GB	~\$5,000+/mo
1B	6.1 TB	~9.2 TB	💀

64 GB → 920 GB = not 15x cost, it's 30-50x operational complexity.



Three real patterns we see teams fall into:

1. "Just use HNSW, it's the best" 80M vectors = 750 GB RAM → ~\$5K/mo + OOM firefighting
2. "Let's add Pinecone alongside Postgres" Two systems to sync, stale vectors, silent recall drops → +\$2K/mo + sync bugs
3. "We'll optimize later" Index rebuild at 50M = 3 days, no deploys → 1-2 week production freeze

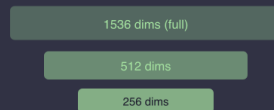
The common thread: these aren't bad tools — they're premature decisions made without doing the math first.

Three Levers Through the RAM Wall

Full precision isn't needed to search. Only to rank the final 10.

1. REDUCE DIMENSIONS

Matryoshka (MRL)



6x smaller

Free. One API parameter.

~97% recall

2. REDUCE BITS

Quantization



4-32x smaller

Needs re-rank step.

~92-99% recall

3. MOVE TO DISK

DiskANN



Graph on disk.
Only tiny index in RAM.

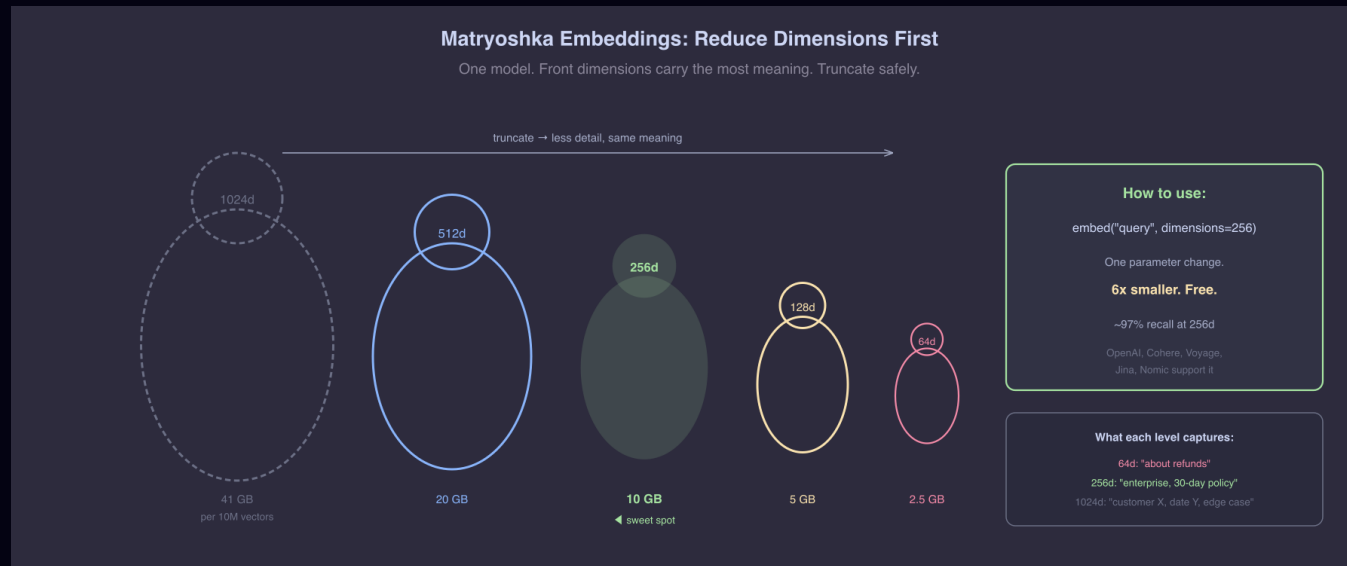
30x less RAM

+5-10ms latency.

~95-99% recall

Compose all three:

1536d FP32: 920 GB → 256d + BQ + DiskANN: <5 GB



FP32 (32-bit) – 32 boxes per value



4 bytes/value

6 GB / 1M

👛 \$5K/mo at 100M



INT8 (8-bit) – 8 boxes per value



1 byte/value

1.5 GB / 1M

4x smaller · ~98%



Binary (1-bit) – 1 box per value



1 bit/value

192 MB / 1M

32x smaller

Memory at 100M vectors:

FP32: 614 GB

INT8: 153 GB

Binary: 19 GB

Search the blurry version fast.
Re-rank with the sharp version for accuracy.

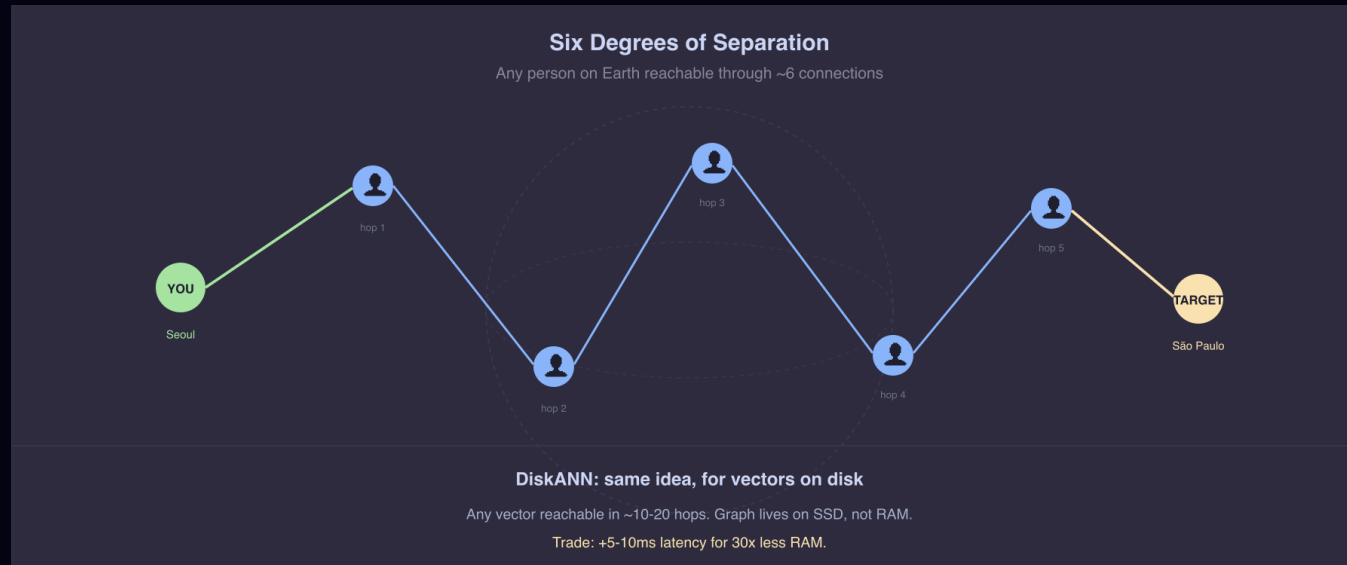
```
python scripts/quantization_demo.py
```

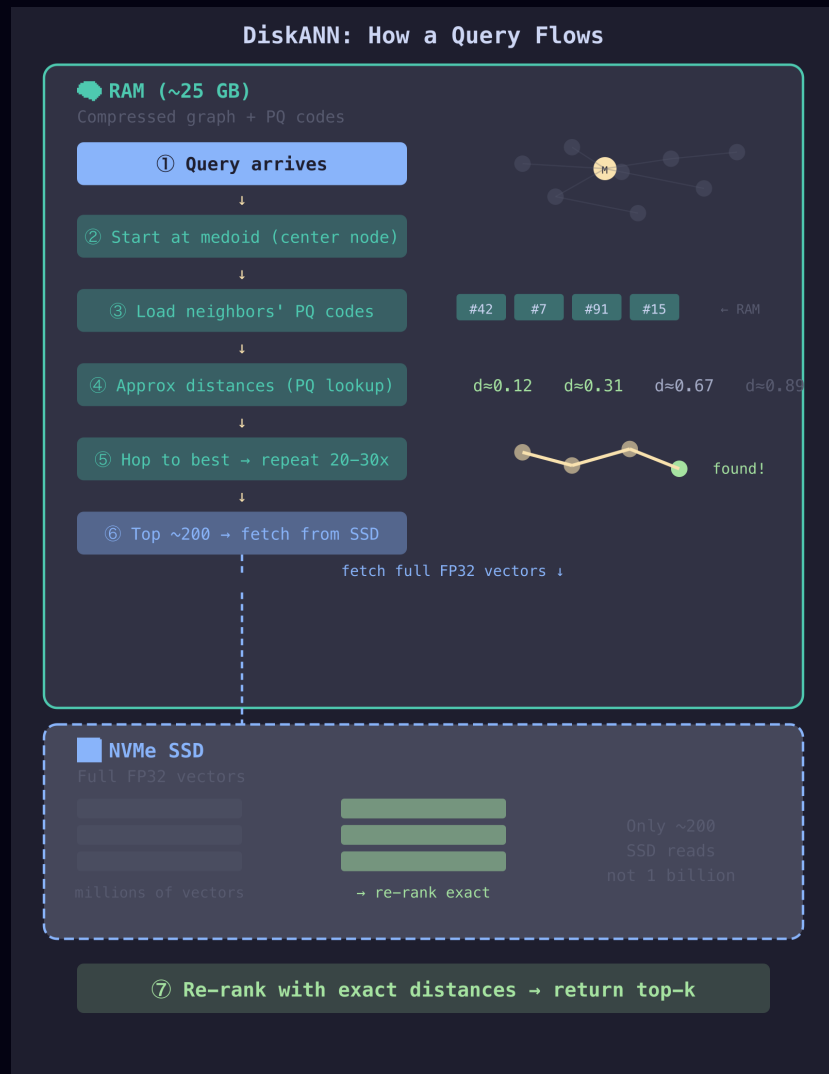
What we just saw:

Method	Index Size (1M)	Recall@10	Notes
FP32 (baseline)	6.1 GB	100%	Exact, expensive
Binary (1-bit)	192 MB	~10%*	Fast but imprecise alone
Binary + re-rank	192 MB + disk	~99%	The production pattern

Why BQ + re-rank works: XOR eliminates 99% of candidates → fetch ~200 full vectors to re-rank.

Recall = "of the true top 10, how many did we actually find?"





	HNSW	DiskANN
RAM (100M)	~920 GB	~10-25 GB
Latency	1-5ms	5-15ms
Cost (100M)	~\$5K/mo	~\$200/mo

How: Every node has short-range + long-range neighbors. Any node reachable in ~10-20 hops.

🚗 → ✈️ → 🚆 → 🚚

Trade: +5-10ms latency for 30x less RAM.

■ Silent Failure #1: Filtered Search

```
SELECT * FROM products
WHERE tenant_id = 42
ORDER BY embedding <=> query LIMIT 10;
```

⚠ The vector index only knows distance.

It's blind to `tenant_id`. Blind to `category`.

HNSW returns the 10 nearest vectors. Then the filter throws 9 of them away.

Result:

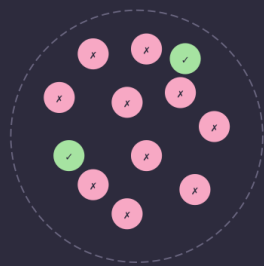
Asked for 10 results. Got 1. Or got 0.

No error. No warning. No log line.

Just an empty page for the user.

Post-Filter (ANN → then filter)

HNSW graph (50K docs)



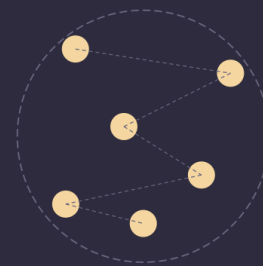
● matches filter ● doesn't match

ANN returns top 10 nearest
→ filter: only 2 match tenant_id=42

✗ Asked for 10, got 2

Pre-Filter (filter → then ANN)

HNSW graph (50K docs)



200 docs match tenant_id=42
→ scattered across graph

✗ Falls back to sequential scan

Both approaches break. You need adaptive filtered search.

Three Fixes for Filtered Search

1. Iterative Scan

Keep scanning until enough match



Scan 40 → not enough → scan 40 more → done

Works for any filter ✓

pgvector 0.8+ baseline

Best for: high cardinality (10K+ values)

2. Partial Index

Separate graph per filter value



One HNSW graph per category

Zero wasted traversal ✓

CREATE INDEX ... WHERE category = 'X'

Best for: low cardinality (2-100 values)

3. Table Partitioning

One partition + index per tenant



Storage-level isolation

Scales to thousands ✓

PARTITION BY LIST (tenant_id)

Best for: multi-tenant SaaS

All three work in pgvector. Iterative scan is the baseline. Add partial/partition for performance.

Filtered Search: What to Use When

Filter Cardinality	Strategy	True filter?
Very low (2-10)	Partial indexes	✓ Yes – separate graph per value
Low (10-100)	Partial indexes	✓ Yes – but many indexes to manage
Medium (100-10K)	Table partitioning	✓ Yes – one partition per tenant
High (10K+)	iterative_scan	✗ No – scans full graph, filters after

This is an active research area.

■ Silent Failure #2: Recall Drift

#1 returns zero results. #2 returns the WRONG results.

```
Month 1: recall@10 = 96% ✓  
Month 6: recall@10 = 84% 📉  
Month 9: "Search is bad" — leadership
```

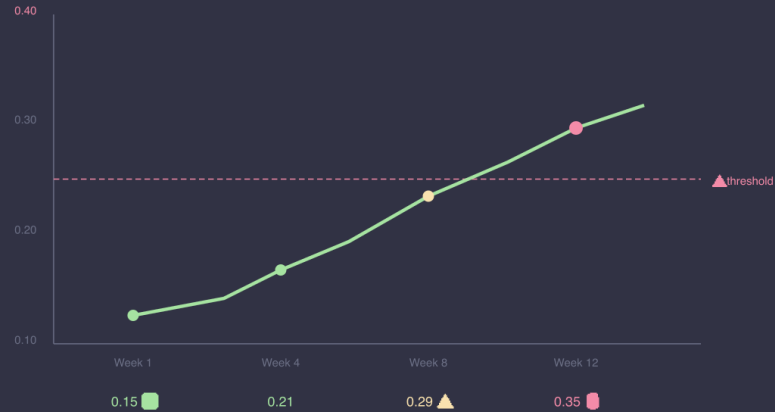
No error. No alert. No log line.

The system doesn't crash. It just quietly becomes useless.

Detecting Drift: Two Monitoring Flows

Canary: Mean Distance to Results

Metric: mean cosine distance | Farther = less relevant

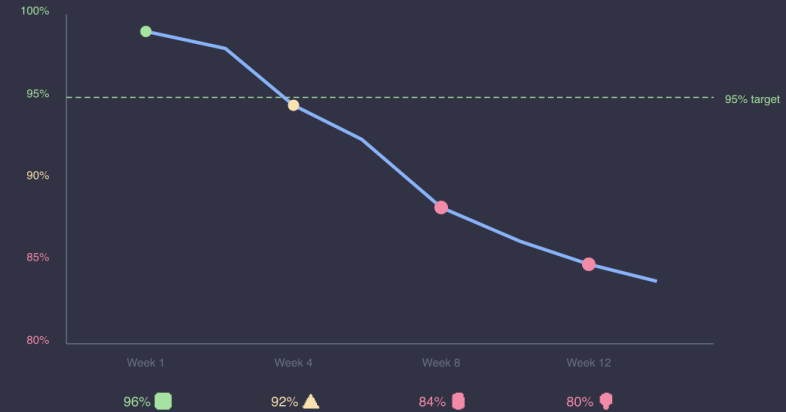


Results drifting away from queries.

Log on every search. No ground truth needed.

Audit: Recall@10 Over Time

Metric: recall@10 | 200 questions with known answers



Quality dropping silently.

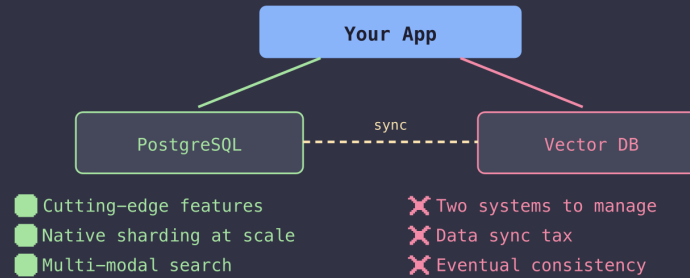
Run weekly. Definitive ground truth.

If you don't measure weekly, you're flying blind.

Specialized Vector DB vs Converged DB

Specialized Vector DB

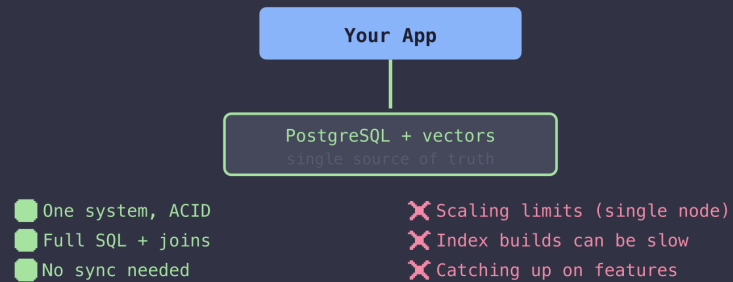
Pinecone, Qdrant, Weaviate



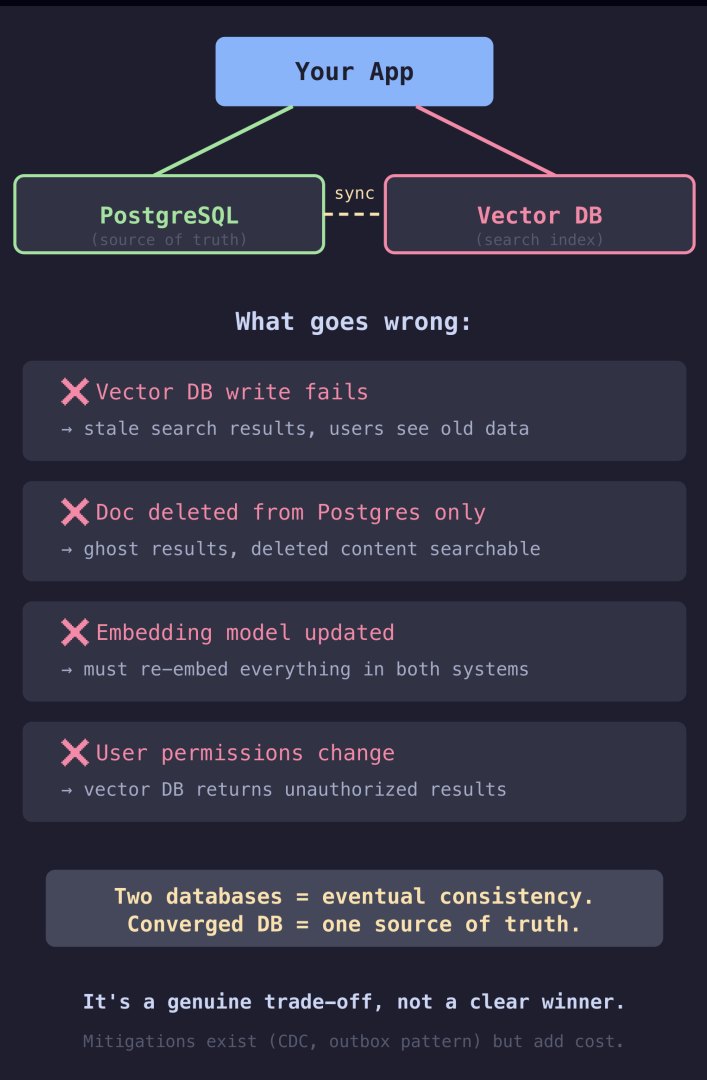
VS

Converged DB

pgvector, Oracle 23ai, Mongo Atlas







■ "But What About 2M-Token Context Windows?"

Gemini has 2M tokens. Claude has 1M. Do you even need retrieval?

	Long Context	RAG
Cost/query	\$0.20-\$2.00	\$0.00008
Latency	20-60s	~1s
Multi-fact recall	~60%	~95%+
Freshness	Reload all	Incremental
Access control	All or nothing	Per-chunk

1,250x cost difference at scale.

■ When to Use Which

Long context wins:

- Static docs < 100K tokens
- One-off analysis
- "What's concerning here?"
- Full-document reasoning

Long context didn't kill RAG. It changed the shape of the decision.

RAG wins:

- Everything at scale
- Interactive latency (<2s)
- Frequent updates
- Multi-tenant access control
- Cited sources

■ Hybrid Search: BM25 + Vectors

Vector search misses exact terms. Keyword search misses meaning.

```
Query: "how to handle payment refund timeout"
```

```
BM25 (keyword): Finds exact "refund timeout"      → precise
```

```
Vector (semantic): Finds "payment reversal logic"  → broader
```

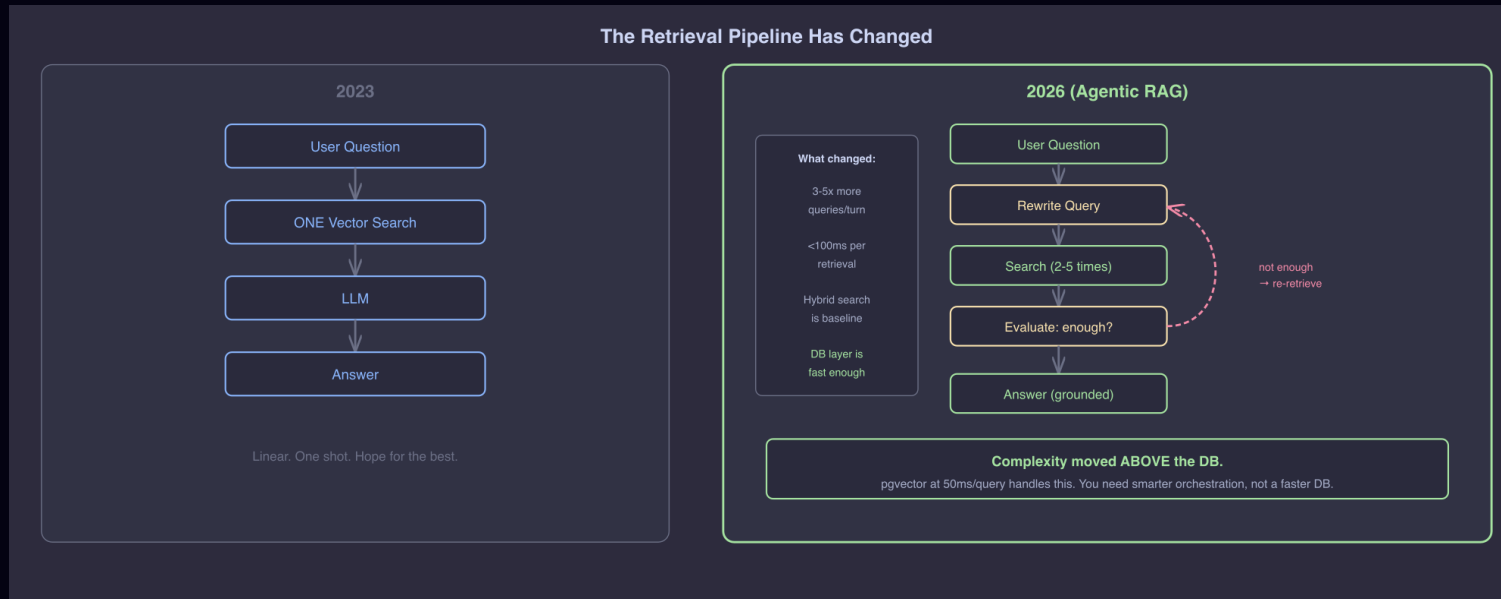
```
Combined: Better recall than either alone.
```

How: Run both searches, merge by rank (Reciprocal Rank Fusion).

pgvector + ParadeDB – both PostgreSQL extensions, same DB.

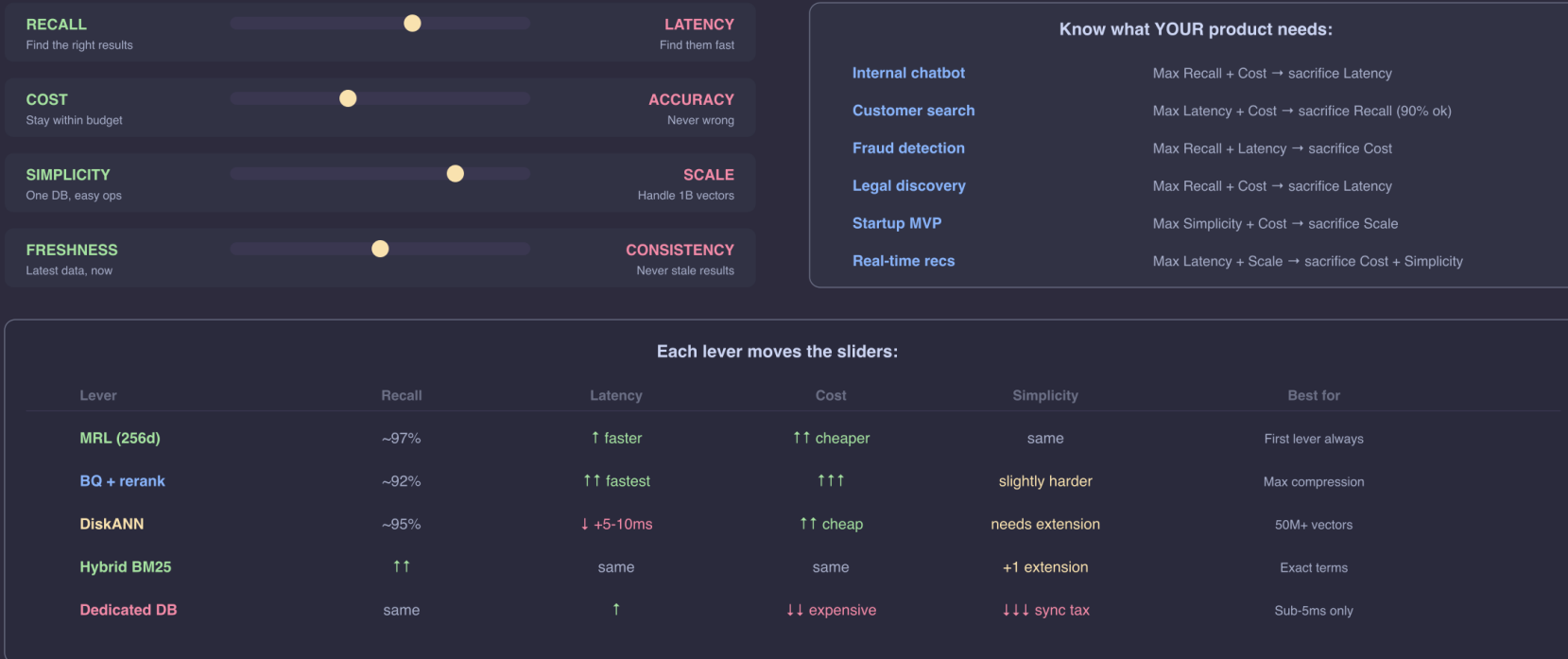
```
CREATE EXTENSION pg_search;  
CREATE INDEX ON docs USING bm25 (content);  
  
SELECT *, paradedb.score(id) FROM docs  
WHERE content @@@ 'payment refund timeout';
```

No separate service. No sync. No eventual consistency. Same transaction.



The Trade-off Mental Model

Every decision is a slider. You can't max all of them.



Decision Matrix: What to Do at Each Scale

SCALE →

< 1M vectors	1M - 10M	10M - 50M	50M - 1B	Sub-5ms @ 100M+
pgvector + HNSW Done. Don't optimize yet. Just ship.	MRL (256d) 6x free savings + iterative_scan for filtered queries	MRL + Quantization halfvec (12x) or BQ (192x) + BM25 hybrid search	DiskANN pgvectorscale Still Postgres. Still open source.	Qdrant Only case for dedicated vector DB

Cross-cutting concerns (at any scale):

Multi-tenant SaaS Partition by tenant Index per partition	Keyword + Semantic ParadeDB extension + RRF Same DB, no sync	Quality Degrading Golden eval set (200 pairs) Measure recall weekly	Agentic RAG (3-5x QPS) Orchestrate above DB DB is fast enough
--	---	--	--

Start with Postgres. Stay longer than you think.

Exhaust: dimensions → quantization → DiskANN → hybrid before adding infrastructure.

All tools open source. Full 30-section decision checklist available as handout.

■ The End

Vectors aren't special. Architecture decisions are. 🚀

Questions?

📧 jeevan.dc24@alumni.iimb.ac.in

🌐 noobj.me

Slides + code + RAG checklist:



```
-- Store truncated prefix (normalize AFTER truncation)
ALTER TABLE docs ADD COLUMN embedding_256 vector(256);
UPDATE docs SET embedding_256 =
  l2_normalize(embedding::real[])[1:256]::vector(256);
CREATE INDEX ON docs USING hnsw (embedding_256 vector_cosine_ops);

-- Cascade: coarse search → fine re-rank
WITH candidates AS (
  SELECT id, embedding FROM docs
  ORDER BY embedding_256 <=> $query_256 LIMIT 200
)
SELECT id, content FROM candidates
ORDER BY embedding <=> $query_full LIMIT 10;
```

Choosing prefix size: Sweep {64, 128, 256, 512} on labeled eval set. Pick smallest holding 95%+ coarse recall@200.



A real query flowing through the cascade:

```
Query: "How does photosynthesis work in deep ocean vents?"

Stage 1 – BM25 keyword search (milliseconds):
  → 1000 docs matching "photosynthesis", "ocean", "vents"
  → includes junk: "ocean pollution", "air vents in buildings"

Stage 2 – Vector ANN with compressed index (milliseconds):
  → narrows to 100 by semantic similarity
  → still includes docs about regular plant photosynthesis

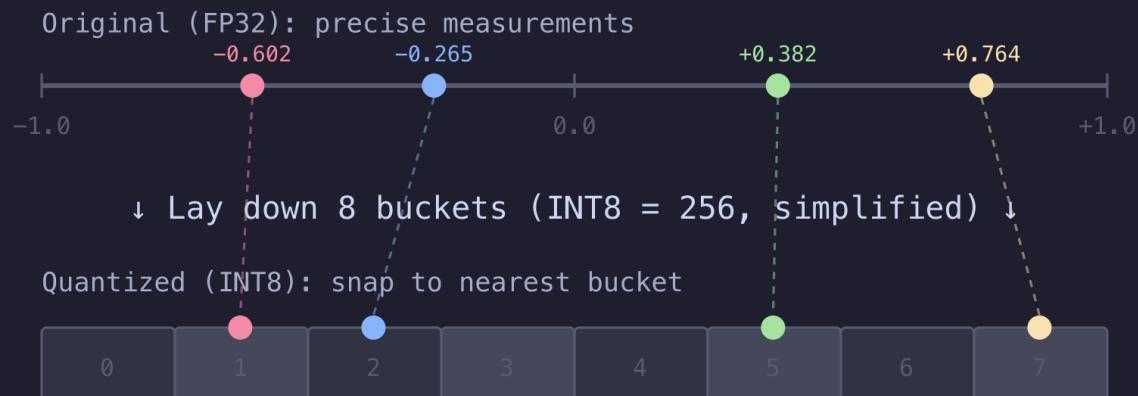
Stage 3 – Cross-encoder re-ranker (tens of milliseconds):
  → reads query + each doc together through a transformer
  → understands "deep ocean" + "vents" = hydrothermal context
  → ranks "chemosynthesis at hydrothermal vents" highest

Return top 10.
```

Bi-encoder encodes query and doc separately — can't see the relationship. Cross-encoder reads them together — every word attends to every word. Cost: per (query, doc) pair → only on ~100 candidates, not millions.

Analogy: Measuring height with a "lazy ruler" that only has 256 notches.

Scalar Quantization: The "Lazy Ruler"



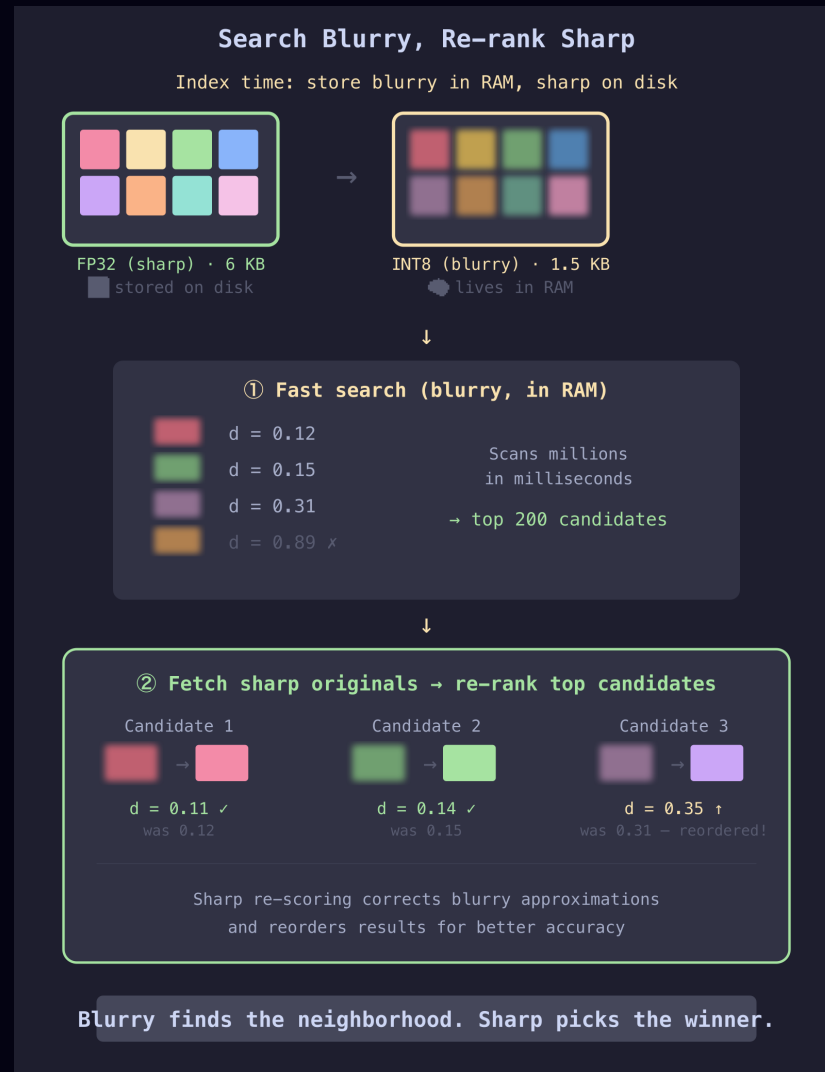
$[-0.602, -0.265, +0.382, +0.764]$

$\rightarrow [1, 2, 5, 7]$

4 bytes per value $\rightarrow$ 1 byte per value = 4x compression

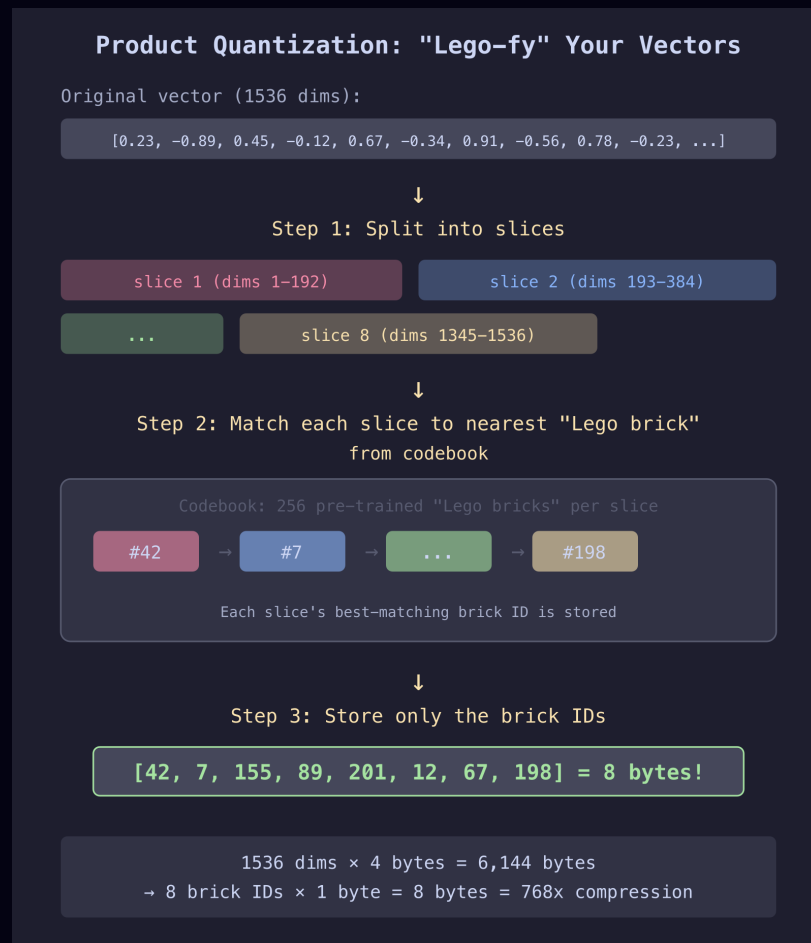
Smart rounding. High-res photo $\rightarrow$ slightly lower-res.

Computers compare integers much faster than long decimals.



Analogy: Instead of storing the whole shape, store a Lego instruction manual.

What's a codebook? A pre-trained dictionary of 256 "representative shapes" (centroids) per slice — built by running k-means on the data. Think of it as a box of 256 Lego bricks.



The clever part — searching without decompressing:

Appendix: How Binary Quantization Works

Analogy: Reducing a photo to pure black and white – no grays.

```
Original:  [+0.23, -0.89, +0.45, -0.12, +0.67, -0.34, +0.91, -0.56]
           ↓      ↓      ↓      ↓      ↓      ↓      ↓      ↓
Rule:      positive=1, negative=0
           ↓      ↓      ↓      ↓      ↓      ↓      ↓      ↓
Binary:    [ 1,    0,    1,    0,    1,    0,    1,    0 ]
```

Comparing two binary vectors – XOR + popcount:

```
Vector A:  1 0 1 0 1 0 1 0
Vector B:  1 1 1 0 0 0 1 1
           - x - - x - - x  ← XOR: 1 wherever they differ

XOR result: 0 1 0 0 1 0 0 1
POPCNT:      count the 1s = 3  ← Hamming distance

Two CPU instructions. No floating-point math at all.
```

Catch: +0.001 and +0.999 both become 1. Massive information loss. Fix: BQ for first pass (top 200), FP32 re-rank (top 10).

Appendix: Quantization Trade-offs

	FP32	FP16 (half)	Scalar INT8	Product (PQ)	Binary (BQ)
Compression	1x	2x	4x	8-64x	32x
Recall (no re-rank)	100%	~99.9%	~95-98%	~85-95%	~75-95%*
Recall (w/ re-rank)	—	—	~98-99%	~95-99%	~92-96%
Speed vs FP32	1x	~1.5x	~2-3x	~5-10x	~15-30x
Training needed?	No	No	No	Yes	No
Best for	Small scale	Easy first win	General purpose	Massive datasets	Speed-critical

MRL + quantization compose: 256d + BQ = 192x compression from 1536d FP32.

Signal	Action
RAM usage > 60% of instance	Evaluate DiskANN
Dataset > 20-50M vectors	DiskANN likely cheaper
Latency budget allows 5-15ms	DiskANN is a fit

```
CREATE EXTENSION IF NOT EXISTS vectorscale;

CREATE INDEX ON docs
  USING diskann (embedding vector_cosine_ops)
  WITH (num_neighbors = 50, search_list_size = 100);

-- Query unchanged – planner picks diskann
SELECT id, content FROM docs
ORDER BY embedding <=> $query LIMIT 10;

-- Tune at query time
SET diskann.query_search_list_size = 150;
```

```
WITH bm25 AS (  
  SELECT id, ROW_NUMBER() OVER (ORDER BY paradedb.score(id) DESC) AS rank  
  FROM docs  
  WHERE content @@@ 'payment refund timeout'  
  LIMIT 100  
,  
vector AS (  
  SELECT id, ROW_NUMBER() OVER (ORDER BY embedding <=> $query) AS rank  
  FROM docs  
  ORDER BY embedding <=> $query  
  LIMIT 100  
)  
SELECT COALESCE(b.id, v.id) AS id,  
       COALESCE(1.0/(60 + b.rank), 0)  
       + COALESCE(1.0/(60 + v.rank), 0) AS rrf_score  
FROM bm25 b  
FULL OUTER JOIN vector v ON b.id = v.id  
ORDER BY rrf_score DESC  
LIMIT 10;
```

k=60 from the RRF paper – dampens high-rank dominance. Only cares about rank position, not raw scores.

Two monitoring flows:

	Live Queries (canary)	Golden Eval (audit)
Frequency	Every query	Weekly
Measures	Distance distribution	Actual recall@10
Ground truth?	No	Yes
Catches	Gross failures fast	Subtle degradation

Minimum viable setup:

1. Log `mean_distance` on every search call
2. 200 labeled query-doc pairs, run weekly
3. Alert if recall drops below 95%
4. CI gate: fail deploy if recall regresses

Open source eval tools: RAGAS, Phoenix (Arize), Langfuse

■ Appendix: Long Context vs RAG

Factor	Favors Long Context	Favors RAG
Corpus size	<100K tokens	>100K tokens
Relevance ratio	>20% relevant	<20% relevant
Latency SLO	Async (45s ok)	Interactive (<2s)
Data freshness	Static	Frequently updated
Query volume	Hundreds/month	Thousands/day
Cost/query	\$0.20-\$2.00	\$0.00008

1,250x cost difference at scale. If any TWO factors favor RAG → use RAG.

References

1. pgvector — github.com/pgvector/pgvector (PostgreSQL License)
2. pgvector scale — github.com/timescale/pgvector scale (Apache 2.0)
3. DiskANN — github.com/microsoft/DiskANN (MIT)
4. ParadeDB — github.com/paradedb/paradedb (AGPL)
5. Matryoshka Embeddings — arxiv.org/abs/2205.13147
6. pgvector 50M Benchmark — Towards AI (July 2026)
7. RAGAS — github.com/explodinggradients/ragas (Apache 2.0)
8. BGE-Reranker — huggingface.co/BAAI/bge-reranker-v2-m3
9. Embedding Quantization — huggingface.co/blog/embedding-quantization
10. Long Context vs RAG — tianpan.co/blog/2026-04-09
11. ANN Benchmarks — ann-benchmarks.com
12. LangGraph — github.com/langchain-ai/langgraph (MIT)